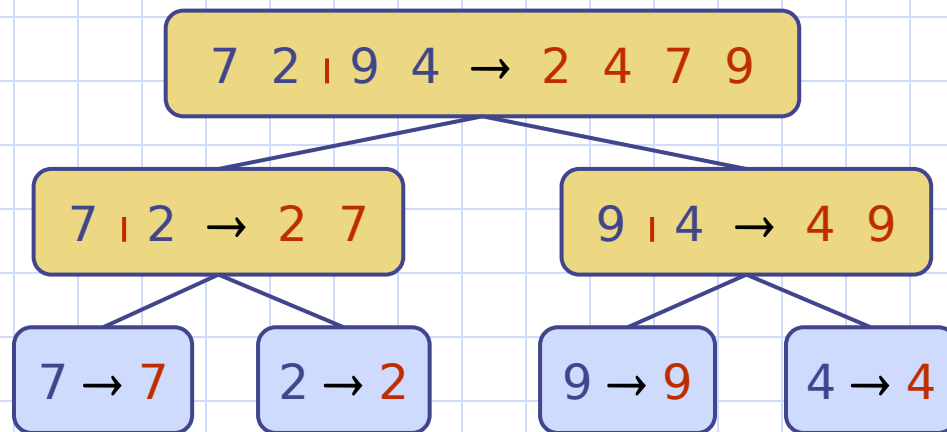


Merge Sort



Divide-and-Conquer

- ◆ **Divide-and conquer** is a general algorithm design paradigm:
 - **Divide**: divide the input data S in two disjoint subsets S_1 and S_2
 - **Recur**: solve the subproblems associated with S_1 and S_2
 - **Conquer**: combine the solutions for S_1 and S_2 into a solution for S
- ◆ The base case for the recursion are subproblems of size 0 or 1
- ◆ **Merge-sort** is a sorting algorithm based on the divide-and-conquer paradigm
- ◆ Like heap-sort
 - It has $O(n \log n)$ running time
- ◆ Unlike heap-sort
 - It does not use an auxiliary priority queue
 - It accesses data in a sequential manner (suitable to sort data on a disk)

Merge-Sort

- ◆ Merge-sort on an input sequence S with n elements consists of three steps:
 - ◆ **Divide**: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
 - ◆ **Recur**: recursively sort S_1 and S_2
 - ◆ **Conquer**: merge S_1 and S_2 into a unique sorted sequence

Algorithm *mergeSort*(S)

Input sequence S with n elements

Output sequence S sorted according to C

if $S.size() > 1$

$(S_1, S_2) \leftarrow \text{partition}(S, n/2)$

mergeSort(S_1)

mergeSort(S_2)

$S \leftarrow \text{merge}(S_1, S_2)$

Merging Two Sorted Sequences

- ◆ The conquer step of merge-sort consists of merging two sorted sequences A and B into a sorted sequence S containing the union of the elements of A and B
- ◆ Merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes $O(n)$ time

Algorithm *merge*(A, B)

Input sequences A and B with $n/2$ elements each

Output sorted sequence of $A \cup B$

$S \leftarrow$ empty sequence

while $\neg A.isEmpty() \wedge \neg B.isEmpty()$

if $A.first().element() < B.first().element()$

$S.addLast(A.remove(A.first()))$

else

$S.addLast(B.remove(B.first()))$

while $\neg A.isEmpty()$

$S.addLast(A.remove(A.first()))$

while $\neg B.isEmpty()$

$S.addLast(B.remove(B.first()))$

return S

Python Merge Implementation

```
1 def merge(S1, S2, S):
2     """Merge two sorted Python lists S1 and S2 into properly sized list S."""
3     i = j = 0
4     while i + j < len(S):
5         if j == len(S2) or (i < len(S1) and S1[i] < S2[j]):
6             S[i+j] = S1[i]           # copy ith element of S1 as next item of S
7             i += 1
8         else:
9             S[i+j] = S2[j]           # copy jth element of S2 as next item of S
10            j += 1
```

	0	1	2	3	4	5	6
S_1	2	5	8	11	12	14	15

i

	0	1	2	3	4	5	6
S_2	3	9	10	18	19	22	25

j

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
S	2	3	5	8	9									

$i+j$

	0	1	2	3	4	5	6
S_1	2	5	8	11	12	14	15

i

	0	1	2	3	4	5	6
S_2	3	9	10	18	19	22	25

j

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
S	2	3	5	8	9	10								

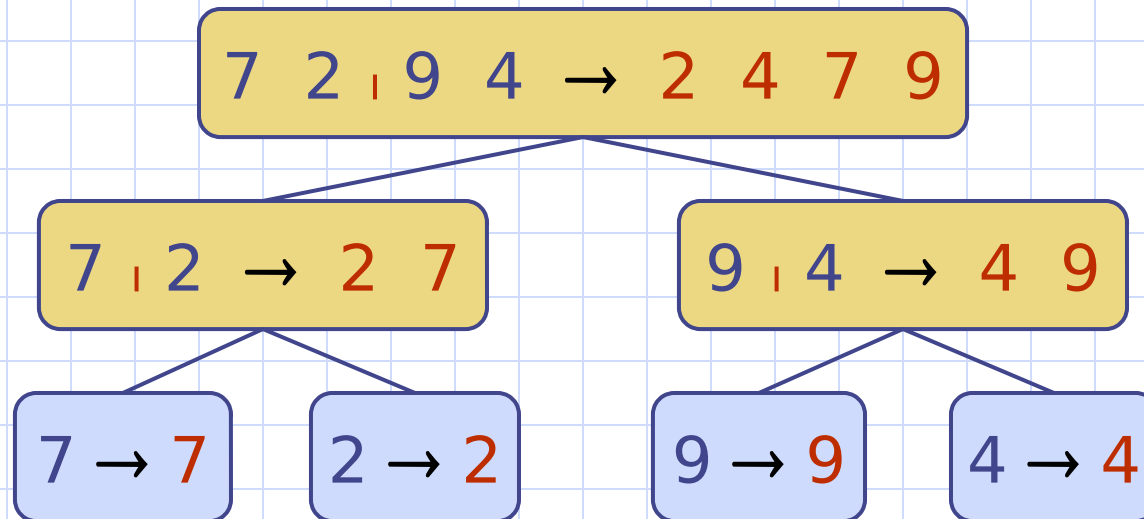
$i+j$

Python Merge-Sort Implementation

```
1  def merge_sort(S):
2      """Sort the elements of Python list S using the merge-sort algorithm."""
3      n = len(S)
4      if n < 2:
5          return                # list is already sorted
6      # divide
7      mid = n // 2
8      S1 = S[0:mid]             # copy of first half
9      S2 = S[mid:n]             # copy of second half
10     # conquer (with recursion)
11     merge_sort(S1)             # sort copy of first half
12     merge_sort(S2)             # sort copy of second half
13     # merge results
14     merge(S1, S2, S)           # merge sorted halves back into S
```

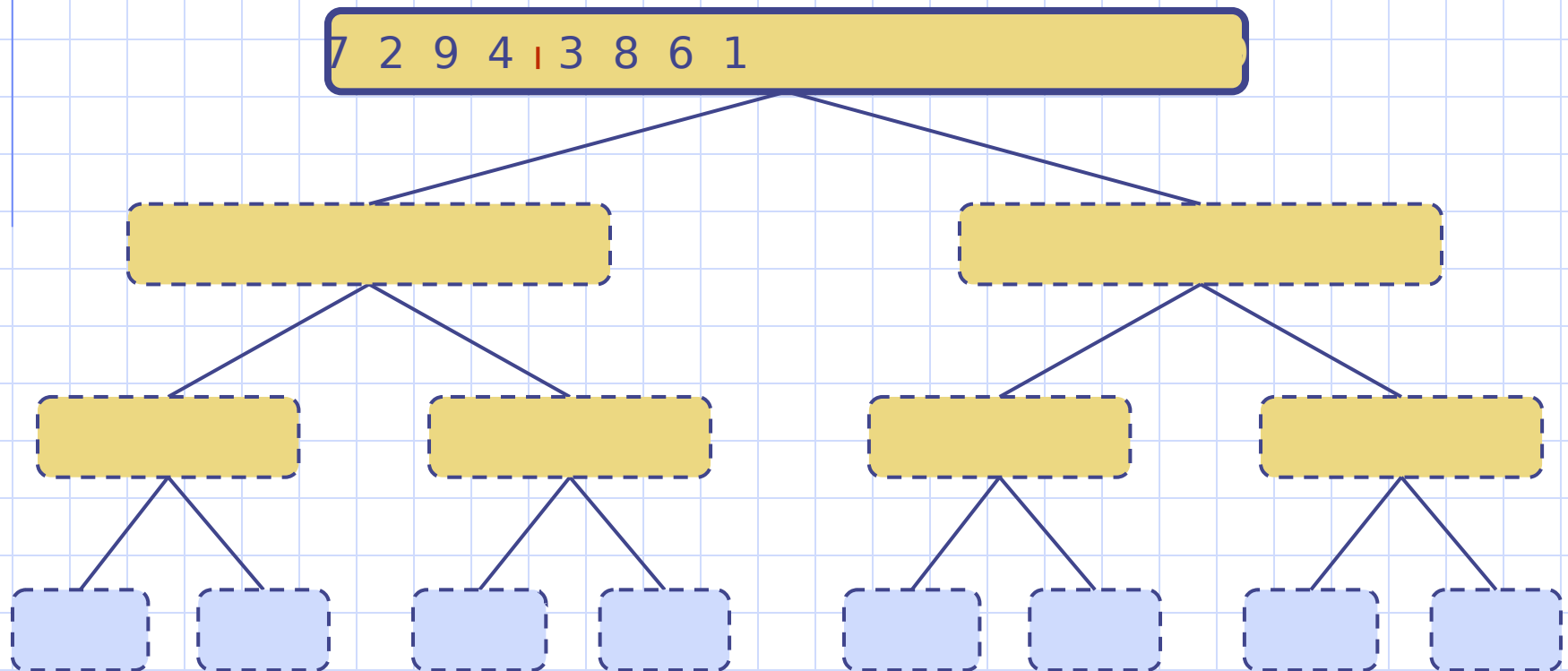
Merge-Sort Tree

- ◆ An execution of merge-sort is depicted by a binary tree
 - each node represents a recursive call of merge-sort and stores
 - ◆ unsorted sequence before the execution and its partition
 - ◆ sorted sequence at the end of the execution
 - the root is the initial call
 - the leaves are calls on subsequences of size 0 or 1

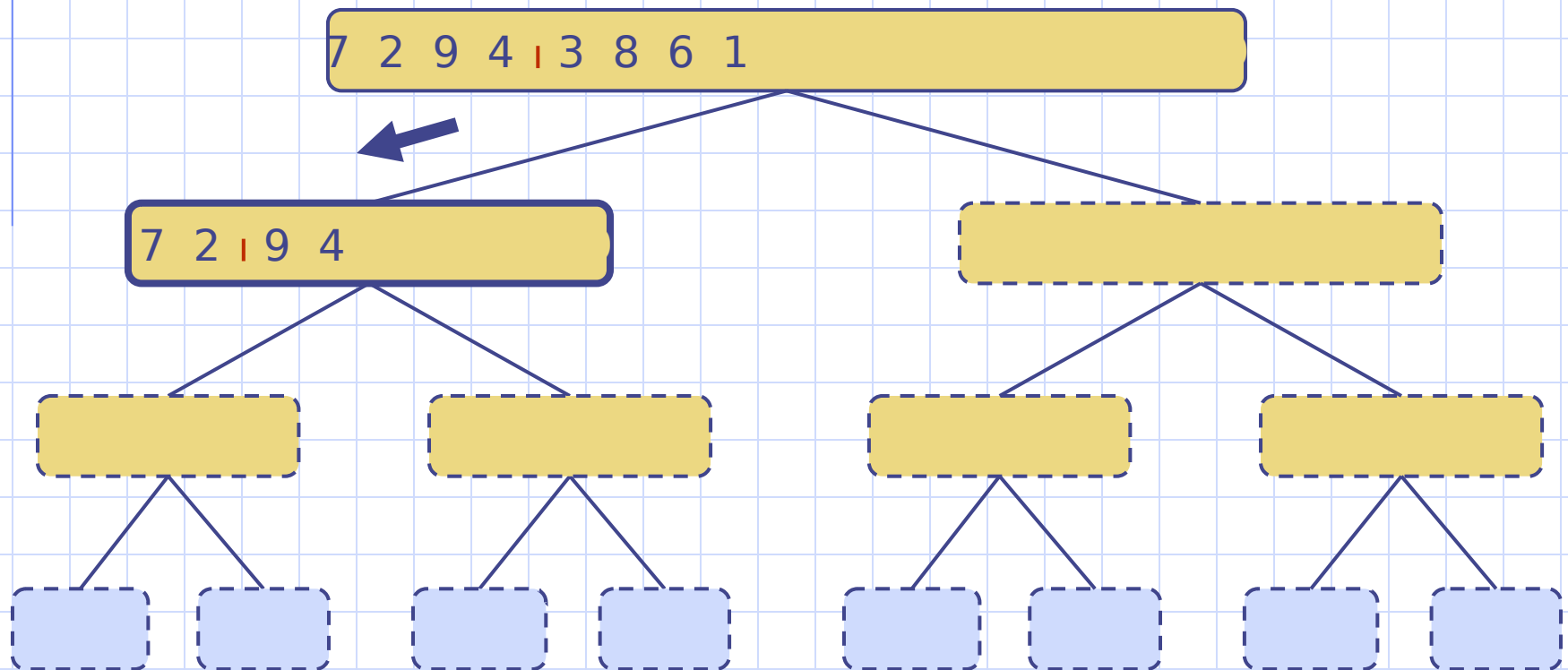


Execution Example

◆ Partition

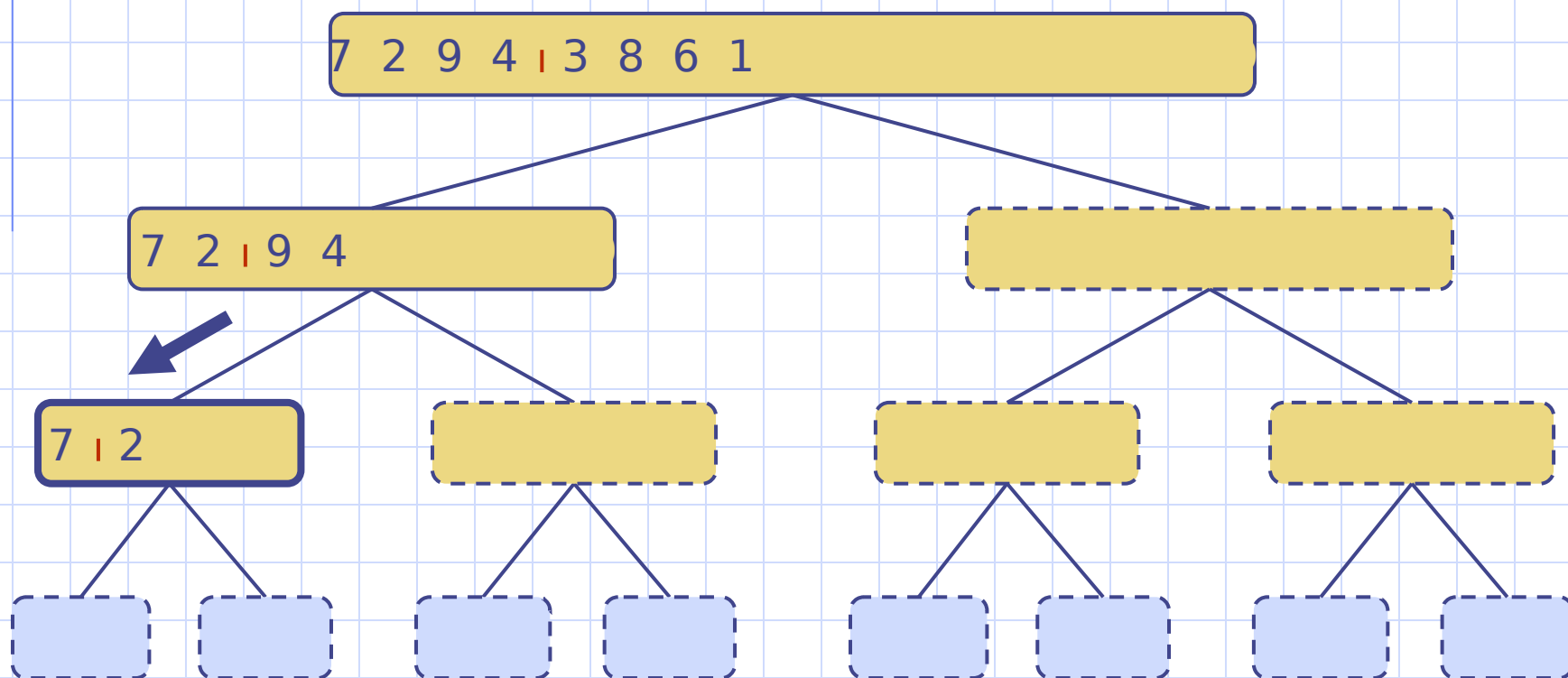


◆ Recursive call, partition



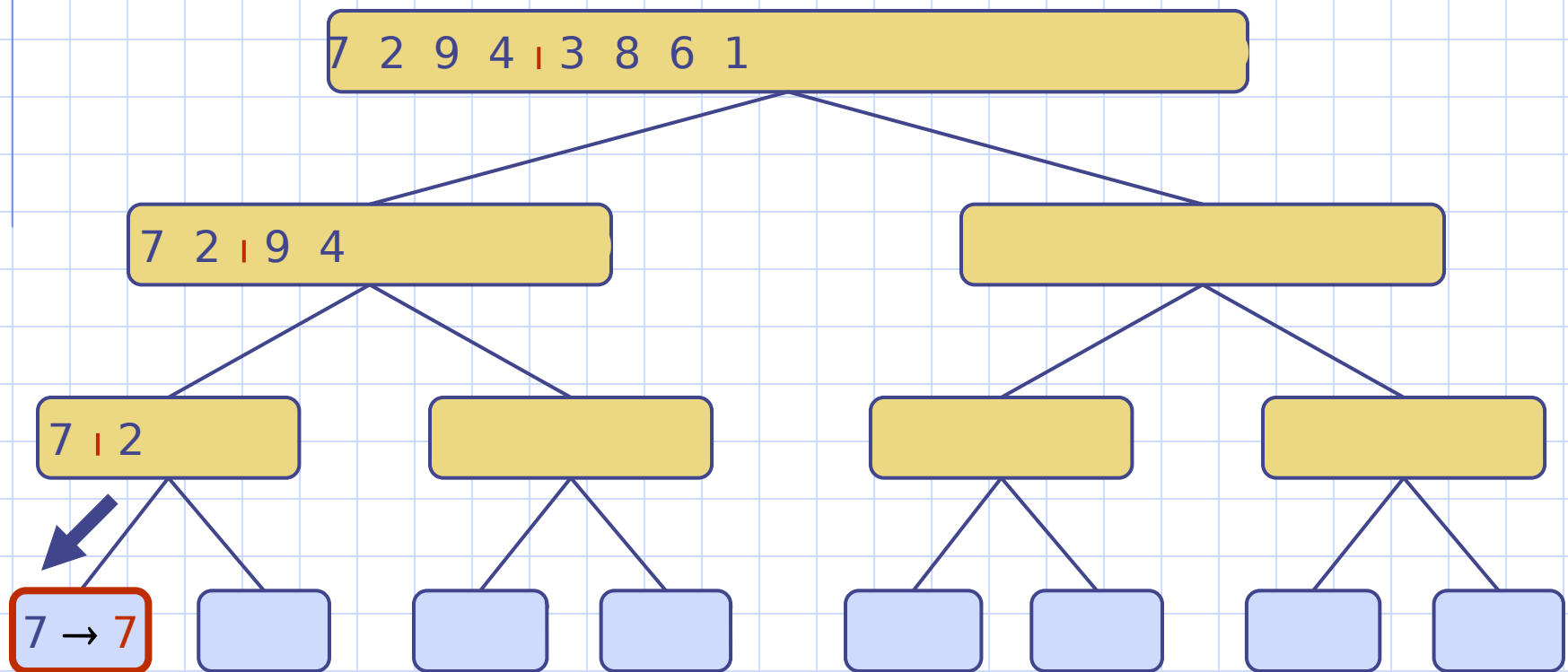
Execution Example (cont.)

◆ Recursive call, partition



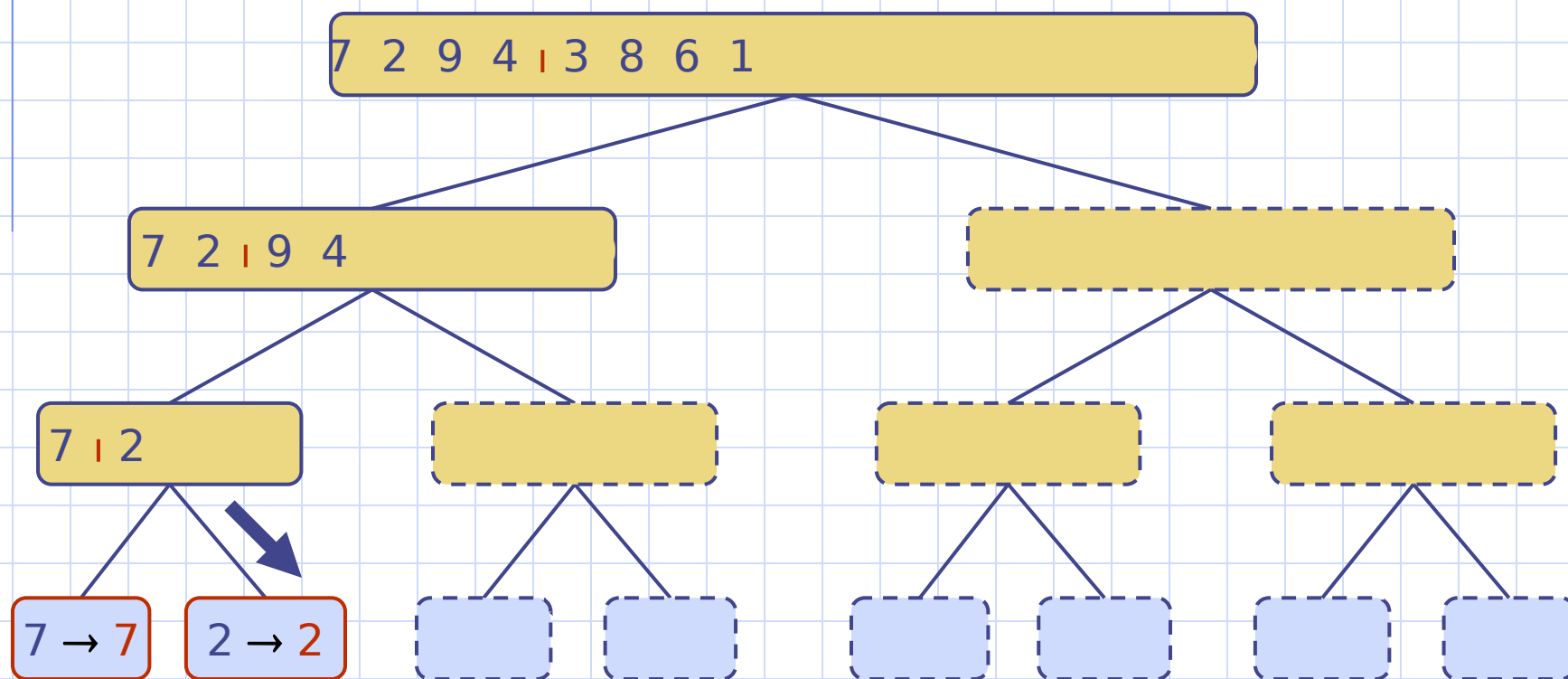
Execution Example (cont.)

◆ Recursive call, base case



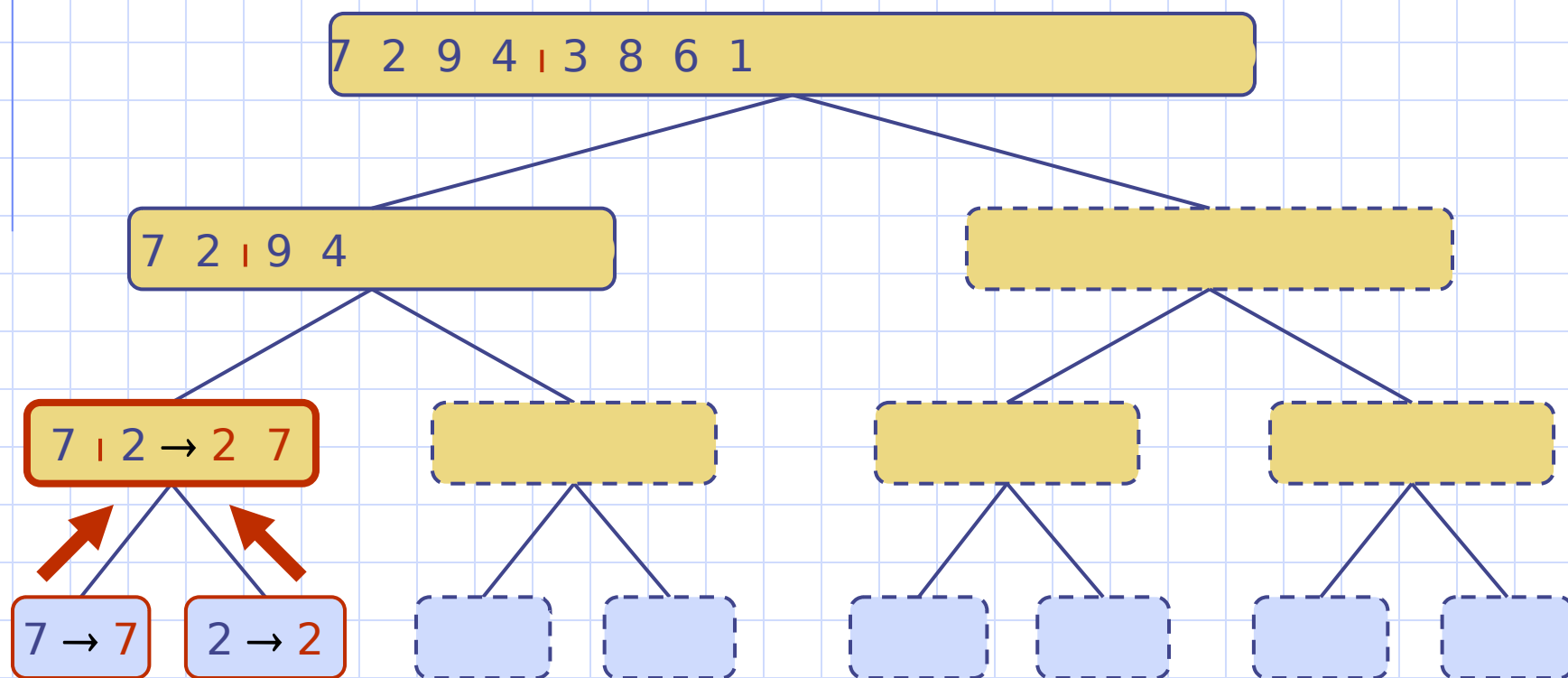
Execution Example (cont.)

◆ Recursive call, base case



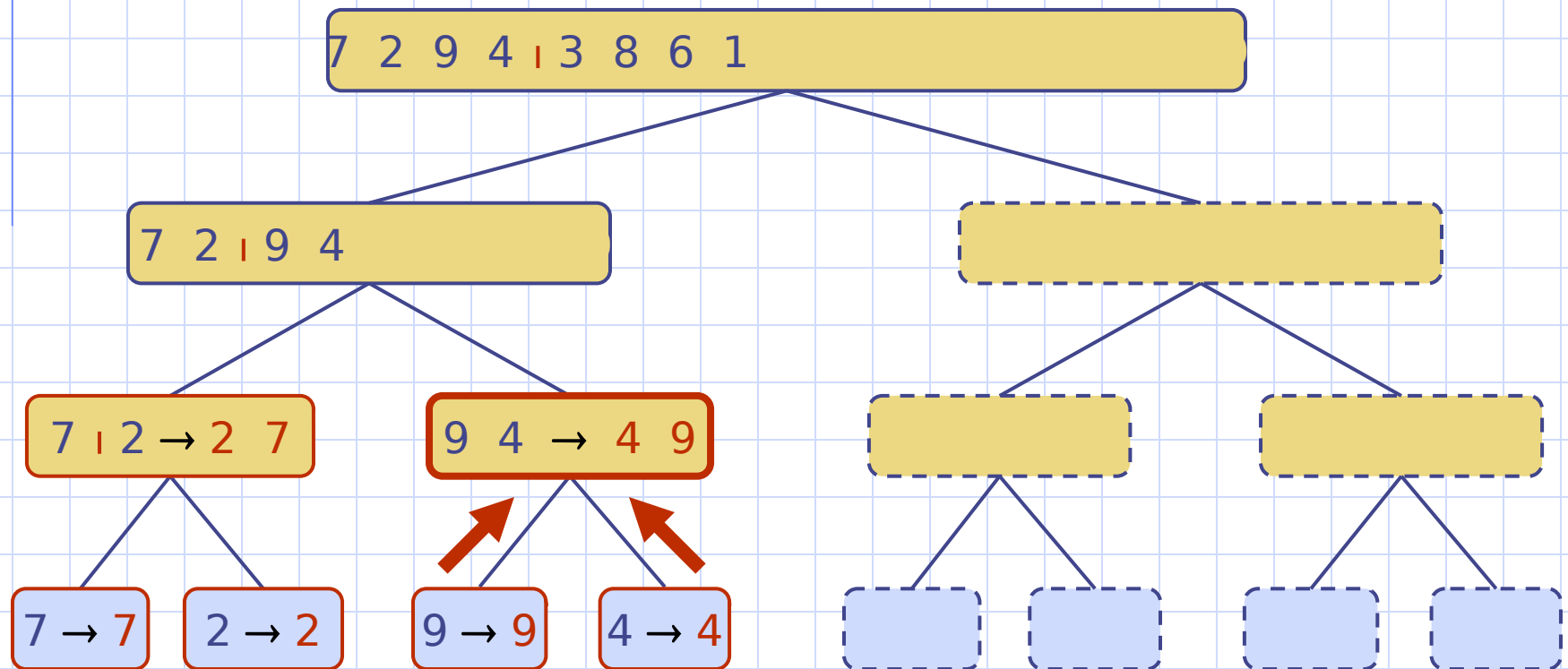
Execution Example (cont.)

◆ Merge



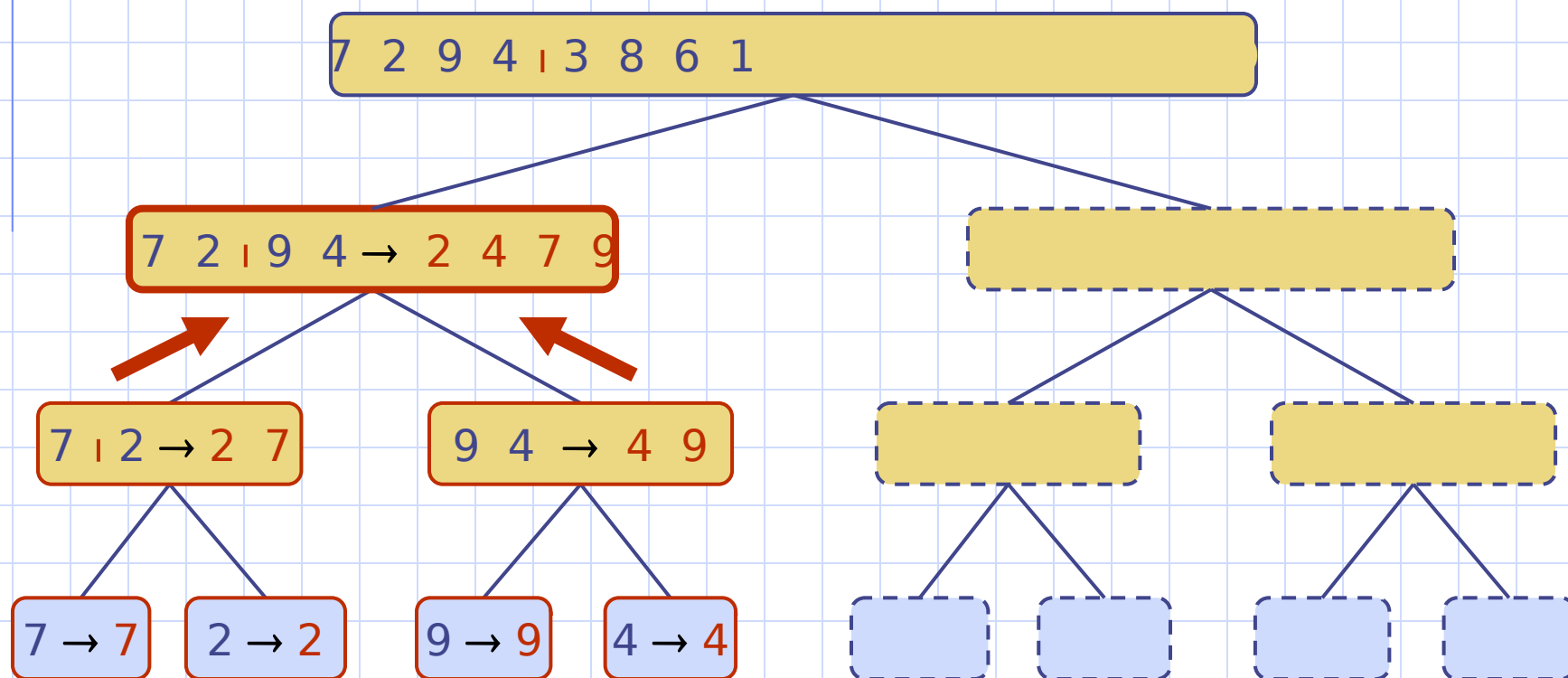
Execution Example (cont.)

◆ Recursive call, ..., base case, merge



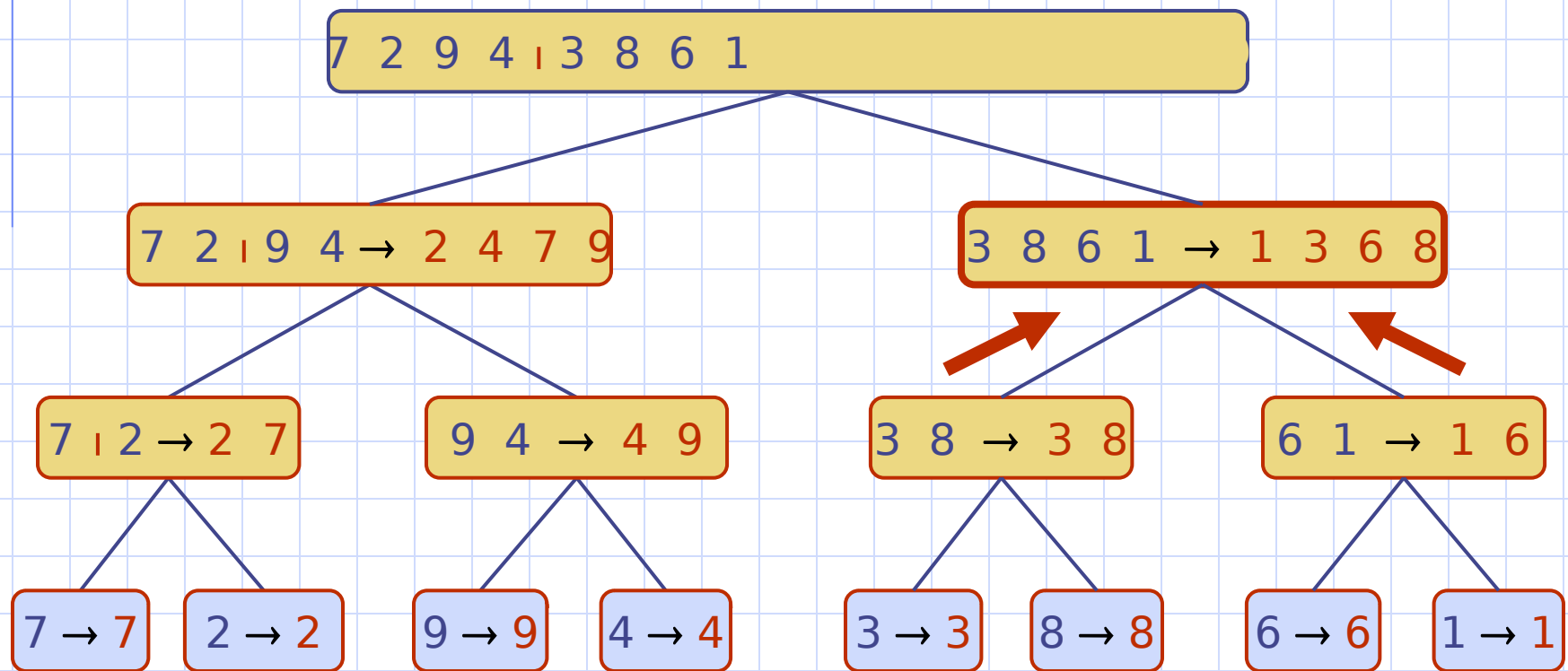
Execution Example (cont.)

◆ Merge



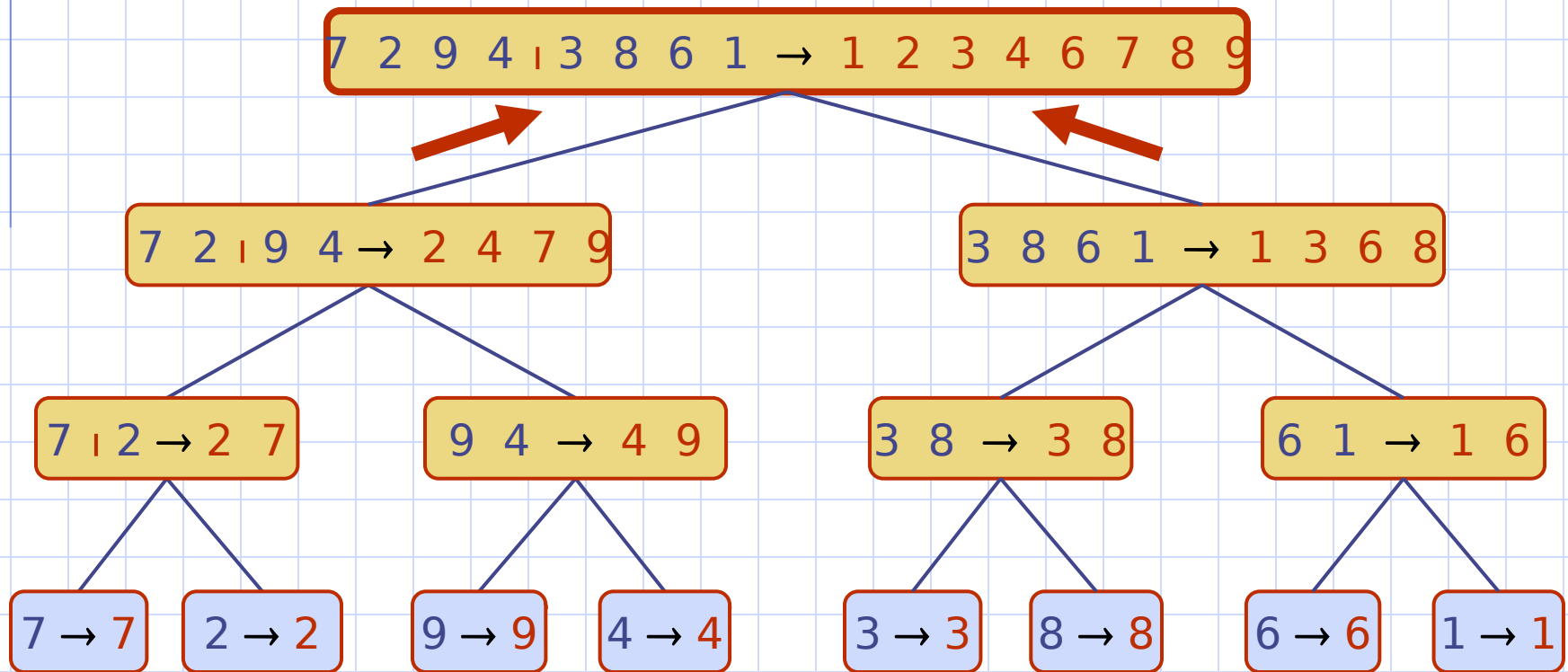
Execution Example (cont.)

◆ Recursive call, ..., merge, merge



Execution Example (cont.)

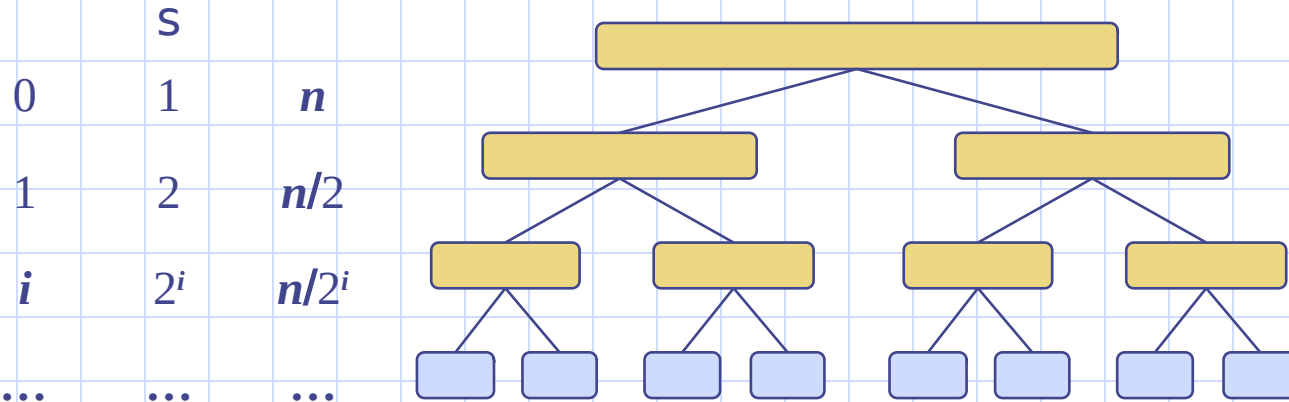
◆ Merge



Analysis of Merge-Sort

- ◆ The height h of the merge-sort tree is $O(\log n)$
 - at each recursive call we divide in half the sequence,
- ◆ The overall amount of work done at the nodes of depth i is $O(n)$
 - ◆ we partition and merge 2^i sequences of size $n/2^i$
 - ◆ we make 2^{i+1} recursive calls
- ◆ Thus, the total running time of merge-sort is $O(n \log n)$

depth #seq size



Summary of Sorting Algorithms

Algorithm	Time	Notes
selection-sort	$O(n^2)$	▪ slow ▪ in-place ▪ for small data sets (< 1K)
insertion-sort	$O(n^2)$	▪ slow ▪ in-place ▪ for small data sets (< 1K)
heap-sort	$O(n \log n)$	▪ fast ▪ in-place ▪ for large data sets (1K — 1M)
merge-sort	$O(n \log n)$	▪ fast ▪ sequential data access ▪ for huge data sets (> 1M)